

A9 · Answer Key and Demo Script

CS425 · Who Are You Really Talking To? · instructor led

INSTRUCTOR COPY

Every transcript below is real output captured on `onyx.boisestate.edu` on October 4, 2026. Boise State's certificate renews every 90 days, so re-run round 2 the morning of class. About 40 minutes: 8, 12, 8, 8, then 4 for the exit question.

Say this at the start

"Every HTTPS page you load answers three questions: was it changed, is the server who it says, and can anyone else read it. Today we watch each answer get made, and then we break them. Everyone `ssh onyx`. Predict before we run anything." Then hand out the sheet.

ROUND 1 · DID ANYONE CHANGE THIS?

```
printf 'pay Bob $10' | sha256sum  then $90, then with -hmac cs425
```

```
51f03ce57e0fd0078b481a668c361fdce4edb7a976d107f297f5e3d3e2e6bcb6  -
04475cefd57cc3e00a47cde385a124f6706381701dabcd72bfd30cc5f1a05096  -
SHA2-256(stdin)= da4f9b2c2f8002ea83506e462fb9c85a5e1516ca9c5ff126375eae65ca91e23a
```

SAY "One character in, a completely different fingerprint out. Now, who can compute that fingerprint?"

Digits. Essentially all of them. A good hash changes about half its bits for any change to the input, so a one character edit gives an unrelated looking hash.

Q1. No: a plain hash proves nothing about who made it. Anyone, Trudy included, can hash the altered message, so Bob sees a matching pair. The MAC hashes the message **with a shared secret**, and Trudy does not have it, so she cannot produce a MAC that matches `$90`. Integrity needs a key. A MAC gives integrity and authentication but not non-repudiation, since both sides hold the secret (section 8.3).

ROUND 2 · WHO VOUCHES FOR WHOM?

```
openssl s_client ... | openssl x509 -noout -subject -issuer -dates -ext subjectAltName  then -showcerts
```

```
subject=CN=boise-state.implem-fe4.ps-pantheon.com
issuer=C=US, O=Let's Encrypt, CN=YR1
notBefore=Aug 25 08:11:11 2026 GMT
notAfter=Nov 23 08:11:10 2026 GMT
    DNS:boise-state.implem-fe4.ps-pantheon.com, DNS:boisestate.edu, DNS:pan-live.boisestate.edu,
    DNS:www.boisestate.edu

 0 s:CN=boise-state.implem-fe4.ps-pantheon.com      i:C=US, O=Let's Encrypt, CN=YR1
 1 s:C=US, O=Let's Encrypt, CN=YR1                 i:C=US, O=ISRG, CN=Root YR
 2 s:C=US, O=ISRG, CN=Root YR                      i:C=US, O=Internet Security Research Group, CN=ISRG Root X1
```

Issuer. Let's Encrypt (intermediate YR1), good for 90 days. Short lives limit the damage from a stolen key, and renewal is automated.

Subject. The hosting company, Pantheon. `www.boisestate.edu` is in the Subject Alternative Name list, and the SAN is what a client matches against.

Q2. Each certificate is signed by the one above it, and Onyx can check each signature with the signer's public key. The chain ends at ISRG Root X1, which the server did not send: it is one of the 147 roots in `/etc/pki/tls/certs/ca-bundle.crt`, shipped by Red Hat with the OS. So Onyx trusts the OS vendor's choice of CAs, and the CAs' checking that the requester controls the name. Certificates move the trust problem to the CA, they do not remove it (section 8.3).

ROUND 3 · FOUR WAYS TO FAIL

```
curl -sS -o /dev/null https://{expired,wrong.host,self-signed,untrusted-root}.badssl.com/
```

```
curl: (60) SSL certificate problem: certificate has expired
curl: (60) SSL: no alternative certificate subject name matches target host name 'wrong.host.badssl.com'
curl: (60) SSL certificate problem: self-signed certificate
curl: (60) SSL certificate problem: self-signed certificate in certificate chain
```

```
$ curl -sS -k -o /dev/null -w '%{http_code}\n' https://expired.badssl.com/
200
```

SITE	WHAT IS WRONG	BROKEN LINK
expired	past notAfter	the leaf's dates
wrong.host	the name is not in the SAN list	the leaf's names
self-signed	the leaf signed itself	no chain at all
untrusted-root	the chain ends at a root Onyx does not have	the trust store

Q3. -k keeps the encryption and drops the **authentication**: you get a private conversation with whoever answered, which could be Trudy in the middle. Confidentiality without authentication is worth very little.

ROUND 4 · WHAT TLS DOES NOT HIDE

```
curl -sv ... | grep then openssl s_client -trace ... | grep -A3 server_name
```

```
* Trying 172.66.147.243:443...
* Connected to example.com (172.66.147.243) port 443 (#0)
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384

    extension_type=server_name(0), length=16
      0000 - 00 0e 00 00 0b 65 78 61-6d 70 6c 65 2e 63 6f  ....example.co
      000f - 6d                                     m
```

POINT AT the right hand column of the hex dump: the hostname, in the clear, in the first message.

Q4. The DNS lookup (normally unencrypted), the server's IP address and port, the hostname in the SNI field, when each connection happens and how long it lasts, and how many bytes go each way. That is often enough to tell what you are doing without reading a word. TLS protects the **contents** (section 8.6).

EXIT QUESTION

```
ssh onyx ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub then, on the laptop, ssh-keygen -F onyx.boisestate.edu -l
```

```
256 SHA256:VX7Yg613MmiIsSzSJRJ3hoyMiAN0thmzQ+RvutdxPyA no comment (ED25519)
# Host onyx.boisestate.edu found: line 46
onyx.boisestate.edu ED25519 SHA256:VX7Yg613MmiIsSzSJRJ3hoyMiAN0thmzQ+RvutdxPyA
```

Typing yes trusted that the very first connection really reached Onyx: **trust on first use**. Nobody vouched for that key, and an attacker on that first connection would have been remembered instead. After that, SSH checks the saved key every time. Onyx never asks: openssl s_client trusts a key because a CA it already trusts signed a certificate binding that key to the name. One model trusts the first contact, the other trusts a third party.

GRADING

Pass/fail, 20 points. Pass when every box has something in it. Wrong predictions are fine, that is what they are for.